

```

import matplotlib.pyplot as plt
import numpy as np
import random
from tabulate import tabulate

def calcular_risco(autonomia):
    """Classifica o nível de risco com base na autonomia."""
    if autonomia > 3:
        return "Baixo"
    elif 1 <= autonomia <= 3:
        return "Médio"
    else:
        return "Alto"

def executar_analise():
    print("--- Sistema de Análise de Abastecimento de Água ---")

    # Entradas básicas [cite: 10]
    try:
        n = int(input("Informe o número de bairros (N): "))
        if n <= 0: raise ValueError
    except ValueError:
        print("Erro: O número de bairros deve ser um inteiro positivo.")
        return

    bairros = []

    # Coleta de dados por bairro [cite: 11-15]
    for i in range(n):
        print(f"\nDados do Bairro {i+1}:")
        nome = input(" Nome: ")
        demanda = float(input(" Demanda diária (m³): "))
        capacidade = float(input(" Capacidade do reservatório (m³): "))
        vol_inicial = float(input(" Volume inicial (m³): "))
        perda_perc = float(input(" Percentual de perda na tubulação (ex: 10 para 10%): ")) / 100
        custo_m3 = float(input(" Custo de bombeamento por m³: "))

        # Cálculos de Processamento [cite: 17-20]
        vol_perda = demanda * perda_perc
        vol_entregue = demanda + vol_perda
        custo_diario = vol_entregue * custo_m3
        autonomia = vol_inicial / demanda if demanda > 0 else 0
        risco = calcular_risco(autonomia)

```

```

bairros.append({
    "nome": nome,
    "demanda": demanda,
    "capacidade": capacidade,
    "vol_inicial": vol_inicial,
    "vol_atual": vol_inicial,
    "perda": vol_perda,
    "entregue": vol_entregue,
    "custo": custo_diario,
    "autonomia": autonomia,
    "risco": risco,
    "historico_vol": [vol_inicial]
})

```

--- SIMULAÇÃO DE 7 DIAS (Desafio Adicional) [cite: 38-41] ---

```
print("\nIniciando simulação de 7 dias...")
```

```
dias = list(range(1, 8))
```

```
for dia in dias:
```

```
    for b in bairros:
```

```
        # Variação aleatória de +/- 20% na demanda [cite: 39]
```

```
        variacao = random.uniform(0.8, 1.2)
```

```
        demanda_dia = b['demanda'] * variacao
```

```
        perda_dia = demanda_dia * (b['perda'] / b['demanda'])
```

```
        necessidade_total = demanda_dia + perda_dia
```

```
        # Atualização do volume (considerando que o reservatório não enche sozinho)
```

```
        b['vol_atual'] -= necessidade_total
```

```
        if b['vol_atual'] < 0: b['vol_atual'] = 0
```

```
        b['historico_vol'].append(b['vol_atual'])
```

```
        # Alerta de situação crítica [cite: 40]
```

```
        autonomia_dia = b['vol_atual'] / b['demanda'] if b['demanda'] > 0 else 0
```

```
        if autonomia_dia < 1:
```

```
            print(f" ! ALERTA: Bairro {b['nome']} em situação CRÍTICA no dia {dia}.")
```

--- SAÍDAS OBRIGATÓRIAS ---

```
# 1. Tabela de Resultados [cite: 34]
```

```
headers = ["Bairro", "Demanda", "Vol. Entregue", "Custo/Dia", "Autonomia", "Risco"]
```

```
tabela = [[b['nome'], b['demanda'], f"{b['entregue']:.2f}", f"R${b['custo']:.2f}",
f"{b['autonomia']:.2f}", b['risco']] for b in bairros]
```

```

print("\n" + tabulate(tabela, headers=headers, tablefmt="grid"))

# Identificação do bairro mais crítico [cite: 37]
bairro_critico = min(bairros, key=lambda x: x['autonomia'])
print(f"\nBairro mais crítico (menor autonomia inicial): {bairro_critico['nome']}")

# 2. Gráficos [cite: 35, 36, 37, 41]
fig, axs = plt.subplots(2, 2, figsize=(15, 10))
plt.subplots_adjust(hspace=0.4)

# A. Demanda vs Volume Entregue
nomes = [b['nome'] for b in bairros]
demandas = [b['demanda'] for b in bairros]
entregues = [b['entregue'] for b in bairros]
x = np.arange(len(nomes))
axs[0, 0].bar(x - 0.2, demandas, 0.4, label='Demanda')
axs[0, 0].bar(x + 0.2, entregues, 0.4, label='Vol. Entregue')
axs[0, 0].set_title("Demanda vs Volume Entregue")
axs[0, 0].set_xticks(x)
axs[0, 0].set_xticklabels(nomes)
axs[0, 0].legend()

# B. Custo Diário
custos = [b['custo'] for b in bairros]
axs[0, 1].bar(nomes, custos, color='green')
axs[0, 1].set_title("Custo Diário por Bairro")
axs[0, 1].set_ylabel("Custo (R$)")

# C. Distribuição de Níveis de Risco
riscos_contagem = {"Baixo": 0, "Médio": 0, "Alto": 0}
for b in bairros: riscos_contagem[b['risco']] += 1
axs[1, 0].pie(riscos_contagem.values(), labels=riscos_contagem.keys(), autopct='%1.1f%%',
colors=['blue', 'yellow', 'red'])
axs[1, 0].set_title("Distribuição de Níveis de Risco")

# D. Evolução do Volume (7 Dias)
for b in bairros:
    axs[1, 1].plot(range(0, 8), b['historico_vol'], marker='o', label=b['nome'])
axs[1, 1].set_title("Evolução do Volume (7 Dias)")
axs[1, 1].set_xlabel("Dia")
axs[1, 1].set_ylabel("Volume (m³)")
axs[1, 1].legend()

```

```
plt.show()
```

```
if __name__ == "__main__":  
    executar_analise()
```